Post-Quantum Cryptography (ML-KEM) Benchmark Report

Performance, Memory, Code Size, and Energy Measurements on the NXP FRDM-MCXN947

By: Zain Ul Haq

Prepared: August 03, 2026

1. Purpose of This Report

Post-quantum cryptography (PQC) algorithms are designed to stay secure even against future quantum computers. Before such an algorithm can be used on small, low-power devices (like the microcontroller used in this thesis), it is important to know exactly how much time, memory, and energy it costs to run. This report presents a set of benchmark measurements for ML-KEM, the algorithm selected by NIST as the new standard for post-quantum key exchange, at all three of its official security levels: ML-KEM-512, ML-KEM-768, and ML-KEM-1024.

The testing approach follows the style used by EEMBC's SecureMark benchmark suite — an established, respected methodology in the embedded-security community — so that these results are structured in a way that is recognizable and comparable to published industry benchmarks. Each measurement (timing, memory, energy) is documented with enough detail that another researcher could reproduce it exactly.

Note: this is not an official EEMBC score. EEMBC's own scoring formula uses committee-decided weightings that are not publicly available, and no official EEMBC benchmark profile currently exists for post-quantum algorithms. The numbers in this report are raw, transparent measurements presented in a similar style — useful and citable on their own, but not a certified EEMBC score.

2. Test Setup

All tests were run on the same hardware: an NXP FRDM-MCXN947 development board, which uses an Arm Cortex-M33 processor running at 150 MHz — a typical low-power microcontroller of the kind used in embedded and IoT devices. Only one processor core was used for these tests (the board has two).

The cryptography code itself comes from PQClean, a well-known, independently audited open-source library of post-quantum algorithm implementations. The plain, portable C version was

used (not a hand-optimized, hardware-specific version), so these results represent an honest, general-purpose baseline rather than a best-case number.

Each of the three algorithm versions (512 / 768 / 1024 — higher numbers mean stronger security, at the cost of more work) was tested by running its three core operations — key generation, encapsulation (the sender's side of a key exchange), and decapsulation (the receiver's side) — 100 times each, and recording:

- Execution time, using the processor's own precise hardware clock-cycle counter.
- Peak memory (stack) usage, using a standard embedded-systems technique that detects the deepest point the memory ever reached during the operation.
- Code size, taken directly from the compiler/linker's own report.
- Energy consumption, measured with a Nordic Power Profiler Kit II (an external, purpose-built current-measurement instrument) attached directly to the board's power-supply circuitry.

All timing and energy measurements were taken while the board ran completely on its own, with no debugger attached — attaching a debugger is known to distort timing and power measurements, so it was avoided for every reported number.

2.1 Hardware and Software Versions

For reproducibility, the exact hardware and software versions used for every measurement in this report are listed below.

Item	Detail / Version
Target board	NXP FRDM-MCXN947 development board
Processor	Arm Cortex-M33, 150 MHz (single core used)
Debug / programming probe	Onboard MCU-Link, firmware v3.172 (CMSIS-DAP)
Energy measurement instrument	Nordic Semiconductor Power Profiler Kit II (PPK2)
Energy measurement point	Board jumper J24 (P3V3_MCU supply rail), Ampere Meter mode
Host PC operating system	Windows 11 Pro
Arm GNU Toolchain (compiler)	14.3.1 (Build arm-14.174)
NXP MCUXpresso SDK	2026.09.00-pvw1
CMake	4.0.3
Ninja (build system)	1.13.2
NXP LinkServer (flashing/debug)	26.6.137
Cryptography source	PQClean project, ML-KEM reference (“clean”) implementation
Energy measurement software	nRF Connect for Desktop — Power Profiler, v4.4.1
PPK2 sampling rate	10,000 samples/second

Analysis scripts	Python 3.12.12

3. Execution Time

The table and chart below show how long each operation takes, on average over 100 runs. As expected, higher security levels take longer — roughly 1.5–1.6× more time at each step up (512 → 768 → 1024), which matches the mathematical scaling of the algorithm and is not a sign of an inefficient implementation.

Security level	Operation	Mean time (ms)	Std. deviation
ML-KEM-512	Key generation	19.9	0.48%
ML-KEM-512	Encapsulation	23.4	0.40%
ML-KEM-512	Decapsulation	28.9	0.33%
ML-KEM-768	Key generation	32.6	0.38%
ML-KEM-768	Encapsulation	38.0	0.33%
ML-KEM-768	Decapsulation	45.4	0.27%
ML-KEM-1024	Key generation	51.0	0.30%
ML-KEM-1024	Encapsulation	57.2	0.27%
ML-KEM-1024	Decapsulation	66.7	0.23%

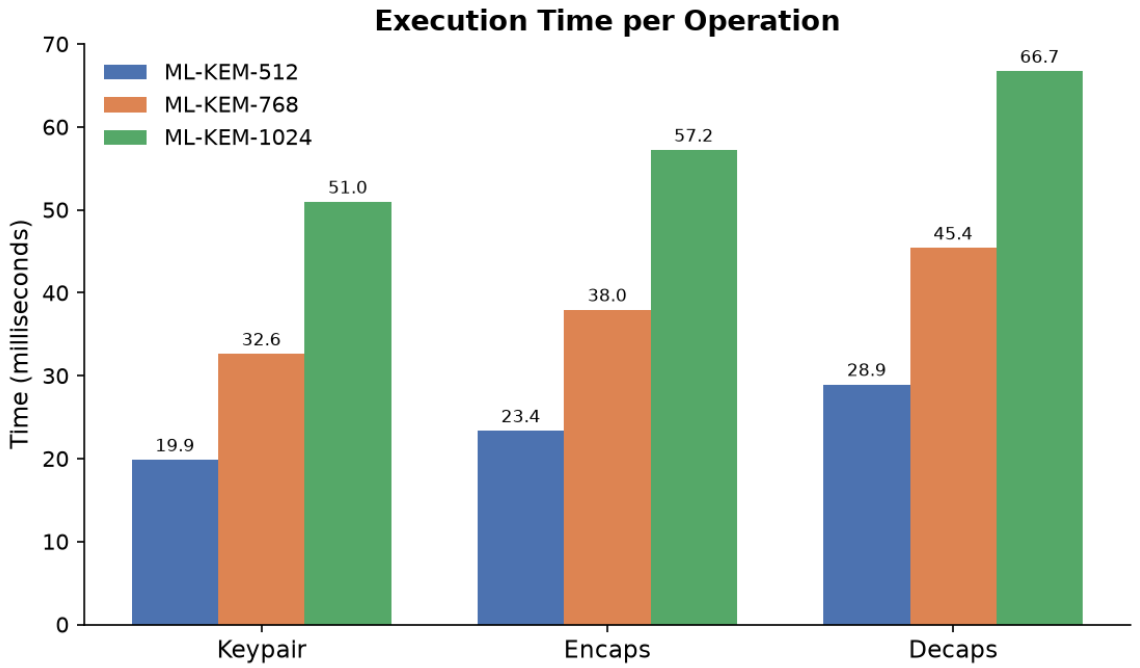


Figure 1: Mean execution time per operation, by security level.

The very low standard deviation (well under 1% of the mean in every case) shows the timing is highly consistent and repeatable — expected for a single-core system with no other software competing for the processor's attention.

4. Memory Usage

Memory usage matters because many embedded devices have only a small amount of RAM available. The chart below shows the peak (worst-case) amount of stack memory each operation ever used while running.

Security level	Operation	Peak stack usage
ML-KEM-512	Key generation	8,960 bytes
ML-KEM-512	Encapsulation	11,592 bytes
ML-KEM-512	Decapsulation	12,344 bytes
ML-KEM-768	Key generation	13,056 bytes
ML-KEM-768	Encapsulation	16,200 bytes
ML-KEM-768	Decapsulation	17,272 bytes
ML-KEM-1024	Key generation	18,176 bytes
ML-KEM-1024	Encapsulation	21,832 bytes
ML-KEM-1024	Decapsulation	23,384 bytes

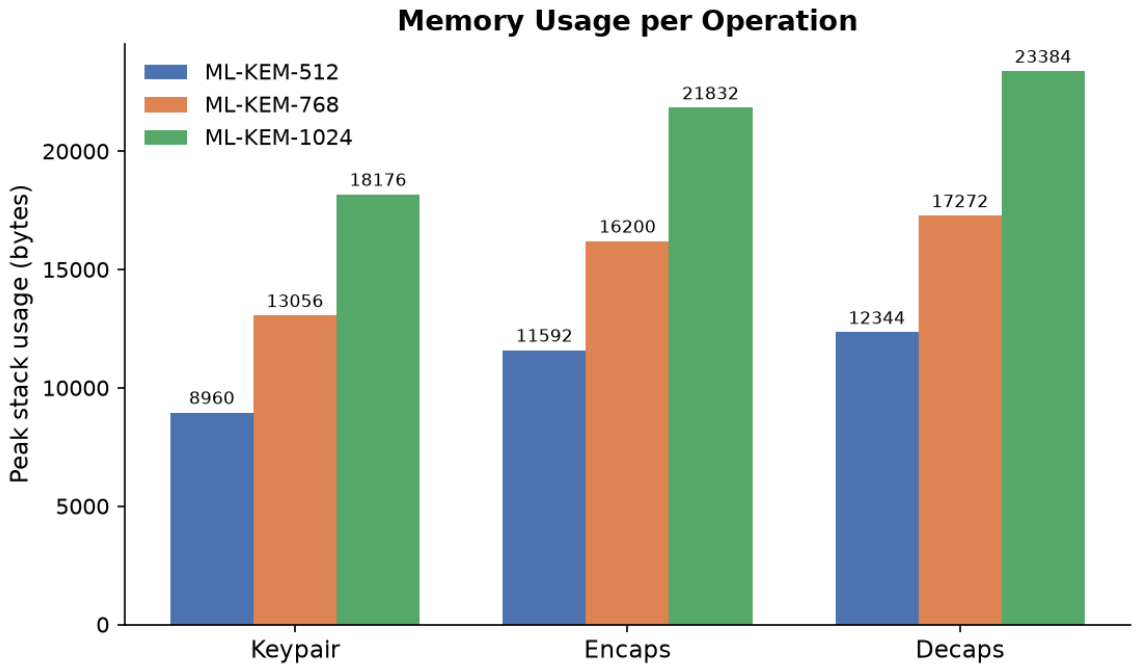


Figure 2: Peak stack (RAM) usage per operation, by security level.

Even the largest case (ML-KEM-1024's decapsulation, at about 23.4 KB) comfortably fits within this board's 32 KB stack budget, leaving roughly 29% headroom. For a smaller target device, this number is the minimum stack space that would need to be reserved.

5. Code Size

Code size determines how much flash storage and static RAM the cryptography library itself takes up, separate from the working (stack) memory used while it runs.

Security level	Flash (code)	RAM (static data)
ML-KEM-512	41,648 bytes	40,928 bytes
ML-KEM-768	41,432 bytes	42,400 bytes
ML-KEM-1024	41,944 bytes	44,032 bytes

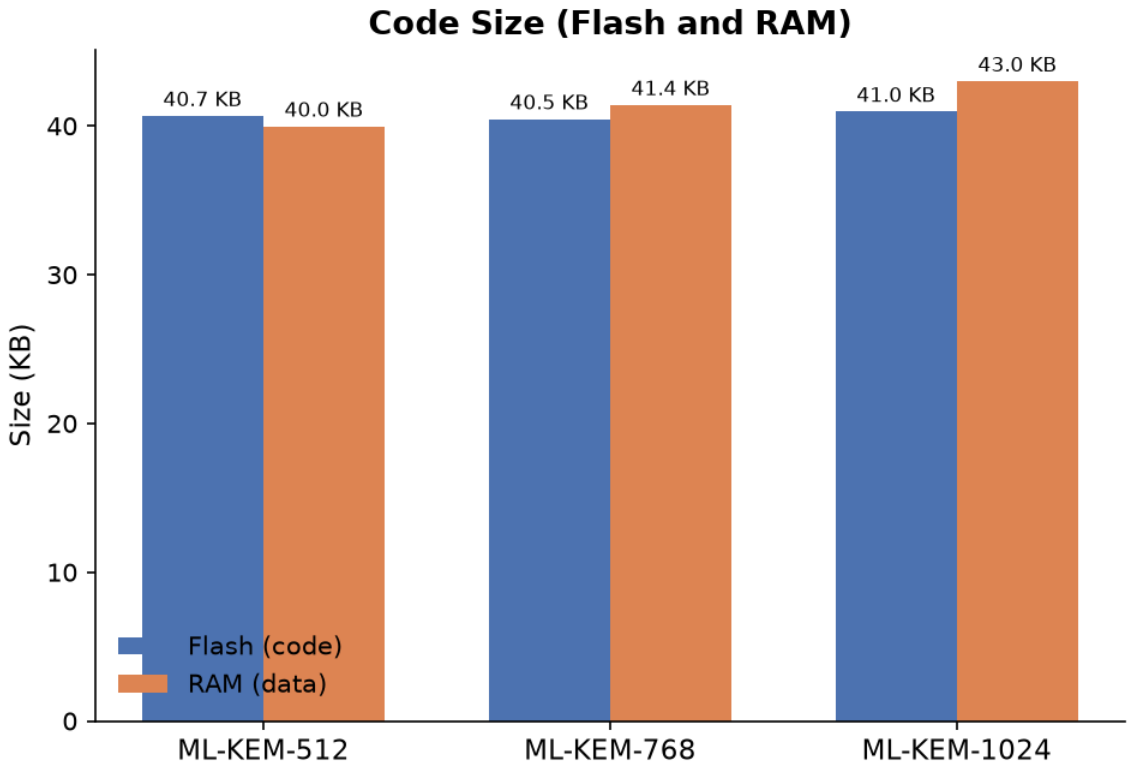


Figure 3: Flash and RAM footprint, by security level.

Flash usage barely changes between security levels (all three variants share the same underlying code), while static RAM grows modestly with each step up — this comes from the larger key and ciphertext buffers that higher security levels require, not from any change in the algorithm's internal working memory.

6. Energy Consumption

Energy was measured directly on the board's hardware using an external current-measurement instrument (Nordic Power Profiler Kit II), wired into the board's power-supply circuit at a point identified from the official board schematic as the main supply for the processor. The instrument records current draw many thousands of times per second; energy is then calculated from that current over time.

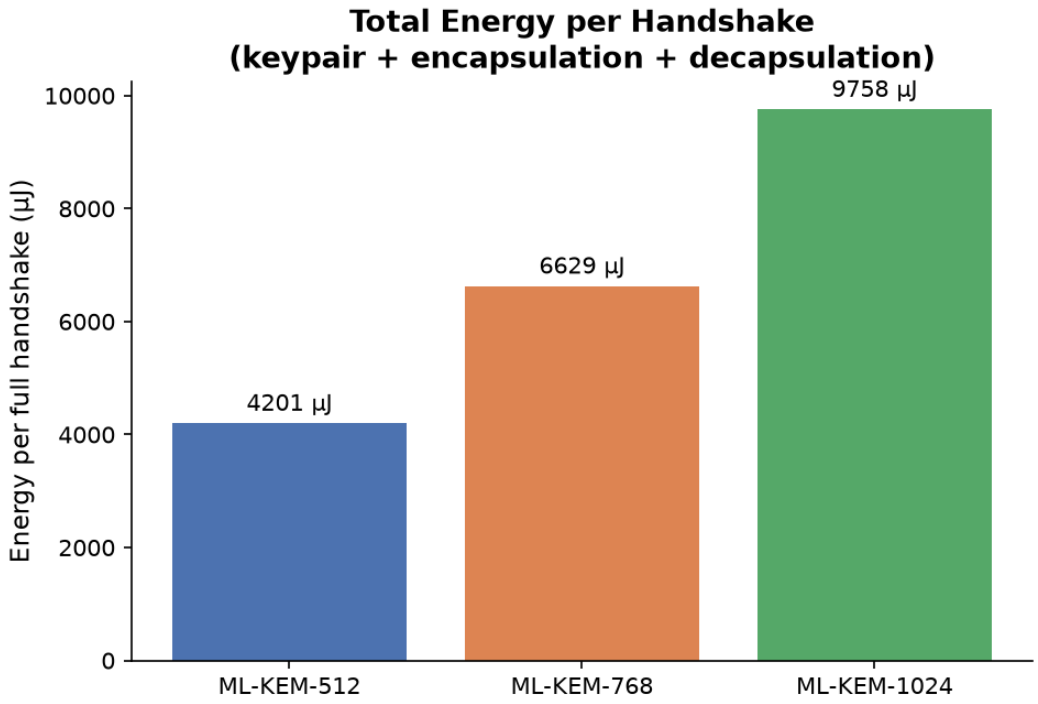


Figure 4: Total energy for one complete key exchange (key generation + encapsulation + decapsulation).

Security level	Energy per full key exchange
ML-KEM-512	~4,201 microjoules (µJ)
ML-KEM-768	6,629 microjoules (µJ)
ML-KEM-1024	9,758 microjoules (µJ)

As with timing, energy use scales up with the security level in a predictable, proportional way — there is no unexpected jump or inefficiency at any security level.

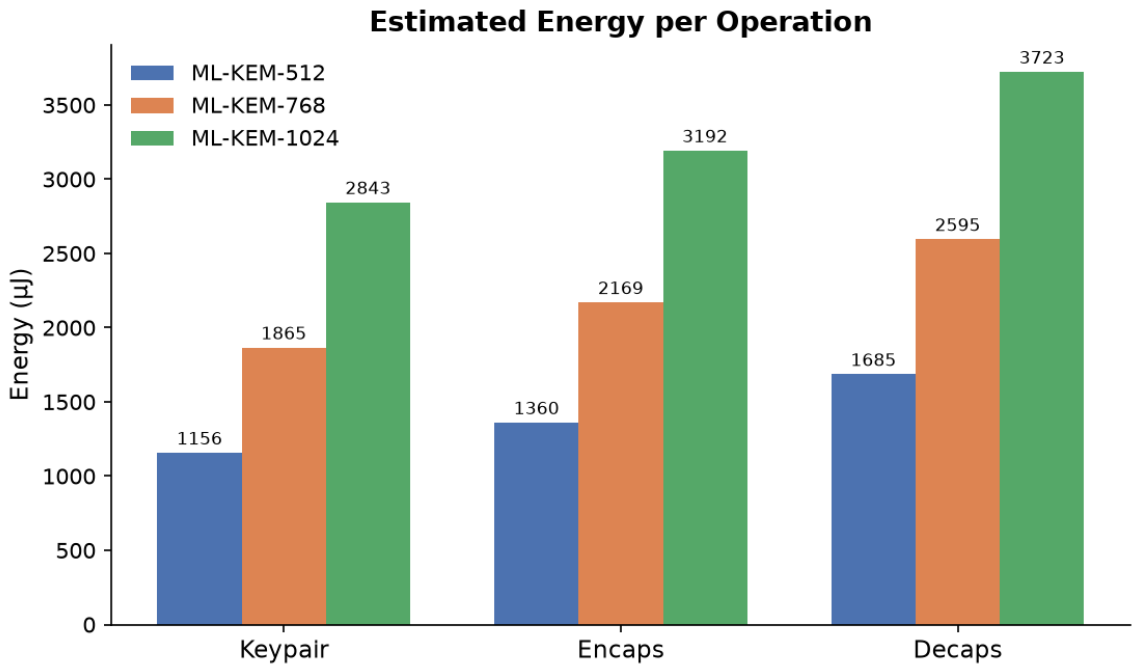


Figure 5: Estimated energy per individual operation.

6.1 Raw Power Waveforms

The charts above are derived summaries. For full transparency, the four figures below show the actual raw current measurements exactly as recorded by the instrument, for every capture used in this report — nothing hidden or pre-processed away. Each plot shows:

- the raw signal itself (light blue), sampled 10,000 times per second — the fine, fuzzy band is real, high-frequency electrical noise from the board's own switching voltage regulator, not measurement error,
- a smoothed trend line (dark blue) to make the underlying pattern easy to see,
- the idle baseline current (dashed line) — what the board draws when not doing cryptography,
- and the shaded “active window” — the period during which all 100 repetitions of key generation, encapsulation, and decapsulation ran back-to-back.

The sharp dip near the start of every trace is the board's power-on reset, captured for completeness — it is excluded from the active window and from all energy calculations.

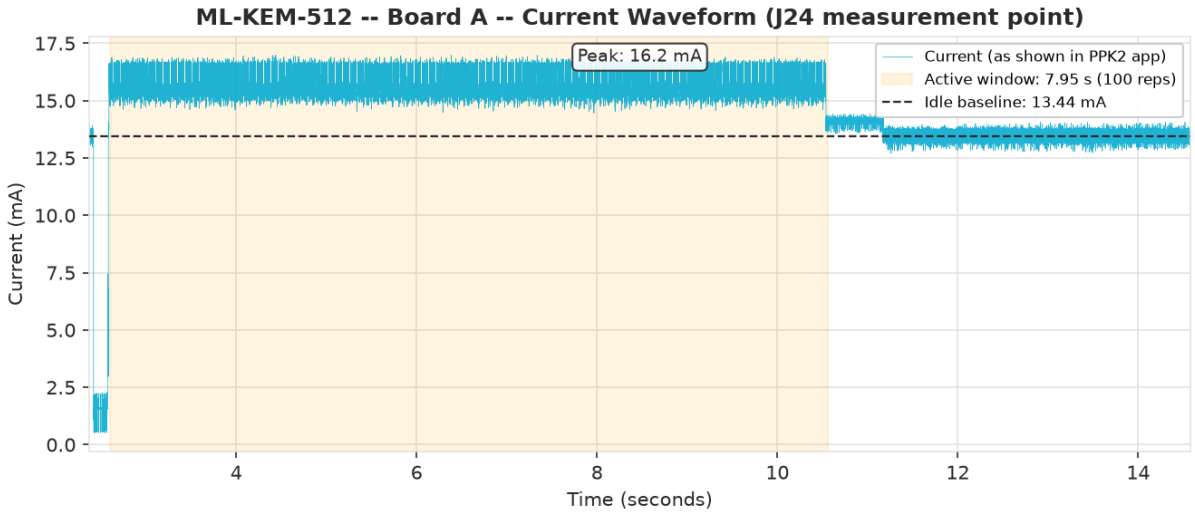


Figure 6: Raw current waveform — ML-KEM-512, Board A.

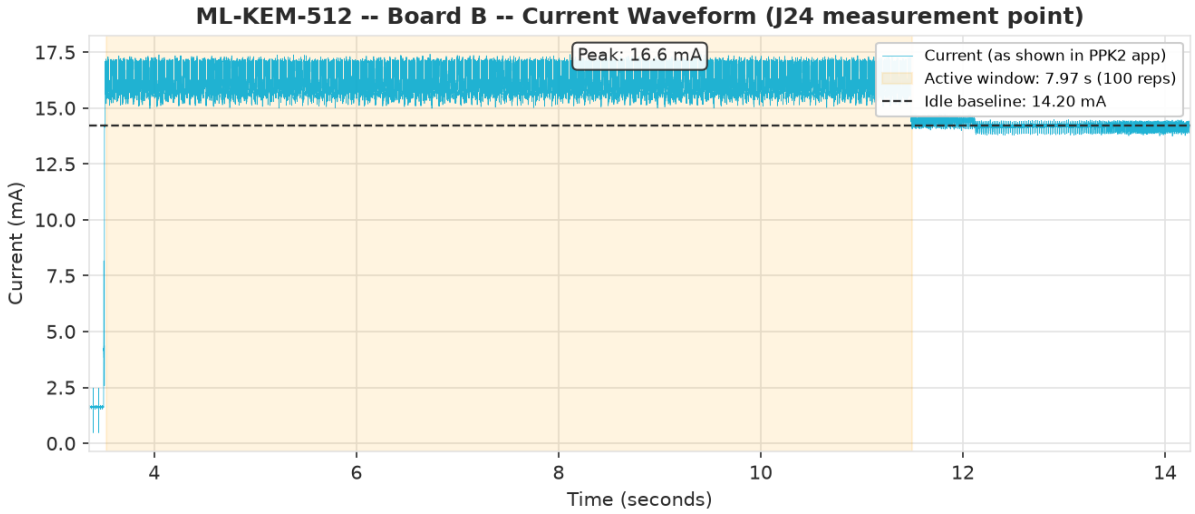


Figure 7: Raw current waveform — ML-KEM-512, Board B (independent repeat, for reproducibility).

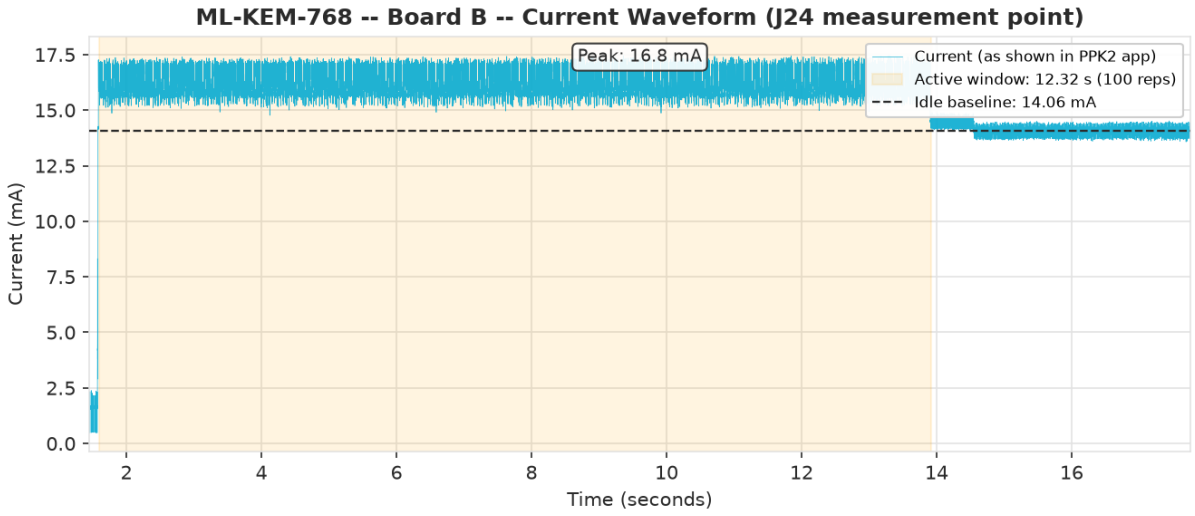


Figure 8: Raw current waveform — ML-KEM-768, Board B.

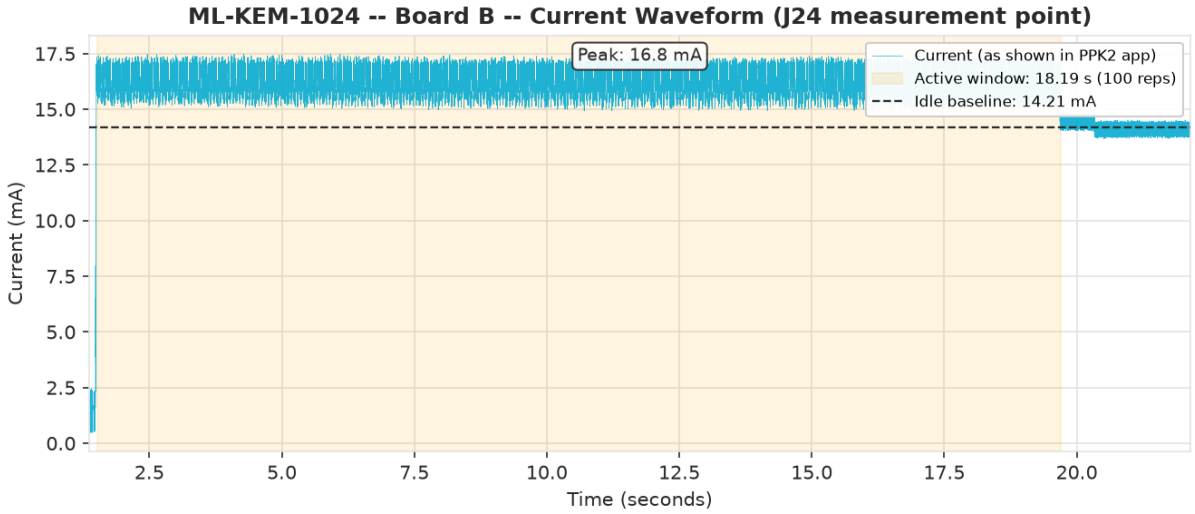


Figure 9: Raw current waveform — ML-KEM-1024, Board B.

Notice the active window visibly grows longer from ML-KEM-512 through ML-KEM-1024 (about 8, 12, and 18 seconds respectively), directly reflecting the extra processing time each higher security level requires — the same trend already seen in the timing results in Section 3.

6.2 How the Energy Numbers Were Calculated

The raw PPK2 recording is a stream of current readings taken 10,000 times per second. Turning that stream into the energy figures in this report involves a few steps:

- The raw signal is smoothed (a short moving average) purely to make it easier to automatically detect where the active measurement period begins and ends — the raw signal's fine electrical noise would otherwise make this boundary hard to detect reliably.

- The idle current level and the point where activity clearly rises above it are used to mark the start and end of the active window (the period covering all 100 repetitions).
- The actual energy total is calculated from the original, unsmoothed current readings across that window — the smoothing is only used to find the boundaries, not to calculate the result itself, so no real measurement data is lost or altered.
- That total is divided by 100 to get the energy of one key exchange, and further divided across the three operations proportionally to their share of processing time, as explained above.

The per-operation breakdown in Figure 5 was calculated by dividing the total measured energy for a full key exchange according to each operation's share of the total processing time (which was measured directly and precisely). This is a very reasonable estimate, since all three operations use the same underlying mathematical building blocks and therefore draw a similar amount of power per unit of time — but, to be fully transparent, it is a calculated estimate rather than a number read directly off separately-measured operations. A follow-up hardware fix (already identified) would allow each operation's energy to be measured completely independently in a future revision of this work.

7. Summary

The table below combines all four measurements into a single overview per security level, using the total time/energy for one complete key exchange (the most realistic real-world scenario) as the headline number for time and energy.

Security level	Full key exchange time	Peak memory	Code + data size	Full key exchange energy
ML-KEM-512	72.2 ms	12,344 bytes	82,576 bytes	~4,201 µJ
ML-KEM-768	116.0 ms	17,272 bytes	83,832 bytes	6,629 µJ
ML-KEM-1024	174.9 ms	23,384 bytes	85,976 bytes	9,758 µJ

Overall conclusion: all three ML-KEM security levels run comfortably within the capabilities of this class of embedded processor. Moving to a higher security level costs roughly 1.5–1.6× more time and energy per step, with only a small increase in memory footprint — a predictable, well-behaved cost curve that makes ML-KEM a practical choice for this kind of hardware.

8. Reproducibility

Every number in this report is backed by raw data files and analysis scripts included alongside this report, so that any of these measurements can be independently re-run and verified step by step.

The included project (folder “mlkem_bench”) is a complete, buildable copy of the firmware used to produce every result in this report. Building it requires the NXP MCUXpresso SDK and Arm GNU Toolchain, installed separately (see Section 2.1 for exact versions) — the easiest way to obtain both is the MCUXpresso for VS Code extension, which can install and manage them automatically.

Once the SDK and toolchain are installed, set two environment variables:

Variable	Value
ARMGCC_DIR	path to your Arm GNU Toolchain install
SdkRootDirPath	path to your MCUXpresso SDK root

Then, from the project folder:

```
cmake --preset debug
```

```
cd debug
```

```
ninja
```

This produces mlkem_bench_cm33_core0.elf in the debug folder. Flash it to the board with LinkServer (or another CMSIS-DAP compatible tool):

```
LinkServer flash MCXN947:FRDM-MCXN947 load mlkem_bench_cm33_core0.elf
```

The project builds ML-KEM-512 by default. To test a different security level, replace the contents of the pqclean folder with the corresponding PQClean reference source (512 / 768 / 1024) and do a clean rebuild of the pqclean build objects before running ninja again.

To build the energy-measurement variant used in Section 6, add -DMLKEM_ENERGY_MODE=ON to the cmake step.